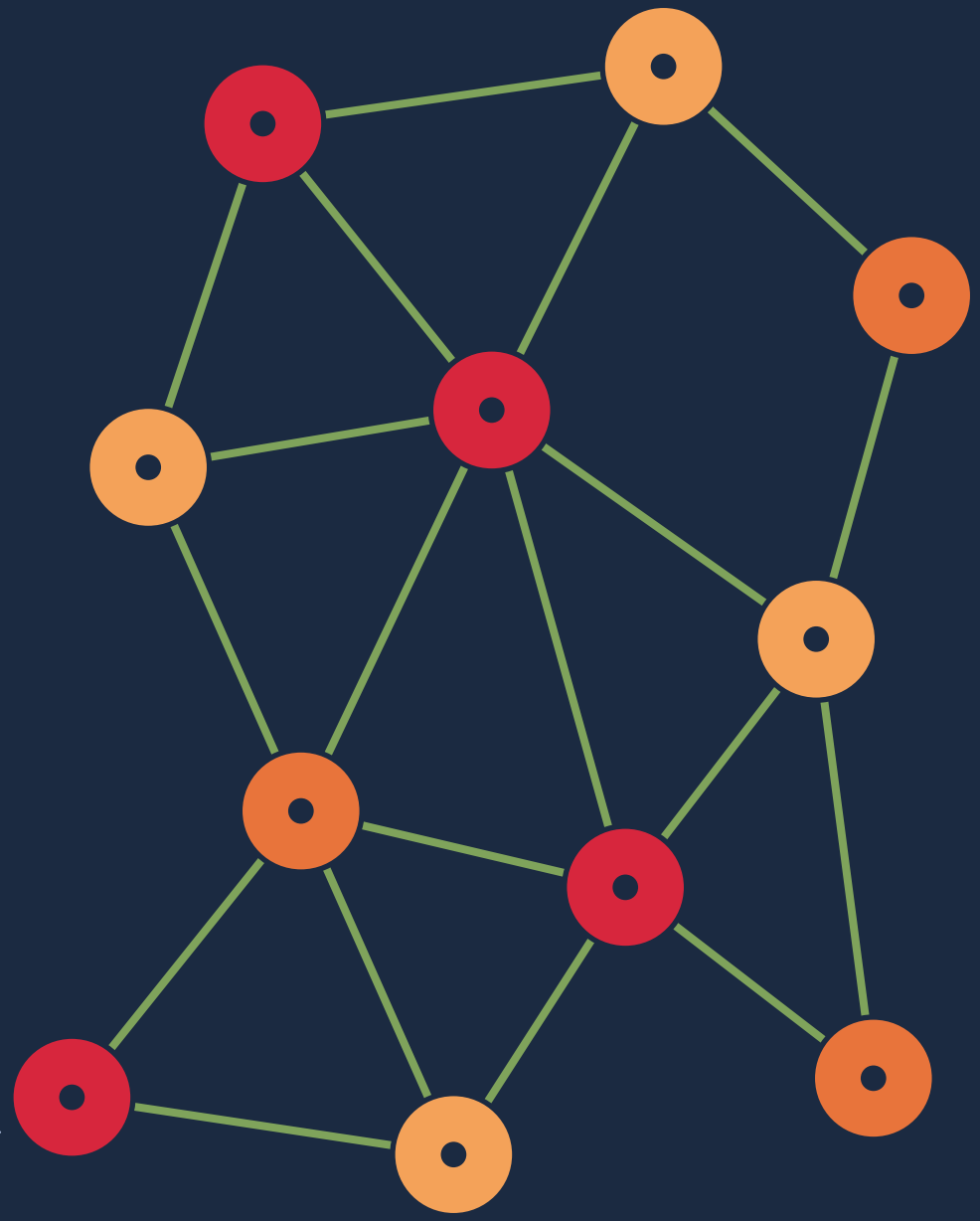
INFORMATION MODELING COURSE · CHAPTER 1

What Is Data Modeling?

Purpose, quality and place in development

What a data model is, why it matters, how to judge a good one, the conceptual–logical–physical stages, and where modeling fits in a project.



Based on G. C. Simsion & G. C. Witt, Data Modeling Essentials, 3rd ed. (Morgan Kaufmann, 2005), Chapter 1

Chapter 1 roadmap

A

Foundations

Sections 1.1–1.3

What a data model is; the data-centered view; a simple example

B

Design and choice

Sections 1.4

Modeling as design: several valid answers

C

Why it matters

Sections 1.5

Leverage, conciseness, data quality

D

A good data model

Sections 1.6–1.7

Eight quality criteria; trade-offs; performance

E

Stages and schemas

Sections 1.8

Conceptual, logical, physical; three-schema architecture

F

Modeling in practice

Sections 1.9–1.15

Methods, people, relevance today, approaches, terminology

Learning outcomes

1. Define a data model and explain why it is a design, not just documentation.
2. Explain why data deserves a central place in system design.
3. Describe leverage, conciseness and data quality as reasons models matter.
4. Evaluate a model with the eight quality criteria and discuss trade-offs.
5. Distinguish conceptual, logical and physical models and the three-schema architecture.
6. Explain where data modeling fits in different development approaches.

CHAPTER 1 · PART A

A

Foundations

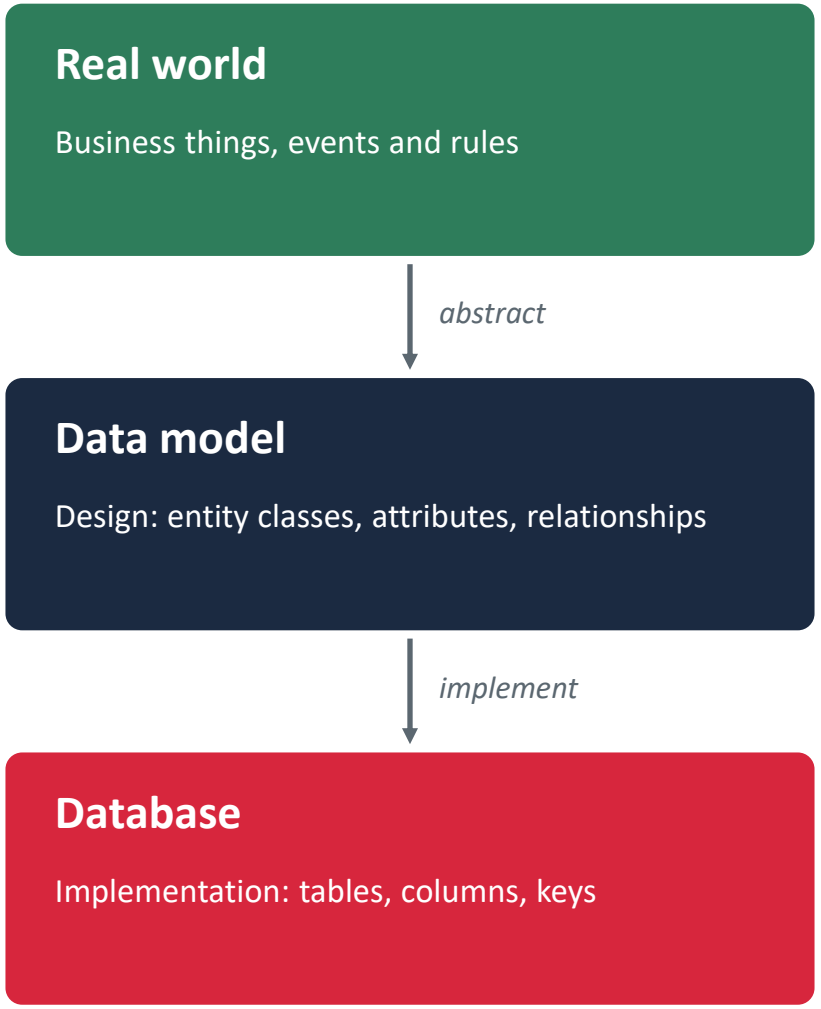
What a data model is and why data comes first

IN THIS PART

- What is a data model
- A design, like a plan
- What it contains
- Data-centered view
- A simple example

What is a data model?

- **Definition:** a description of how data is organized in a database: what is stored and how the pieces relate.
- **It is a design,** not just documentation: a plan for the database, like an architect's plan for a house.
- **Every database has one:** if nobody drew it, it still exists implicitly in the tables someone created.
- **Deliverables:** a diagram plus definitions of entity classes, attributes, relationships and rules.

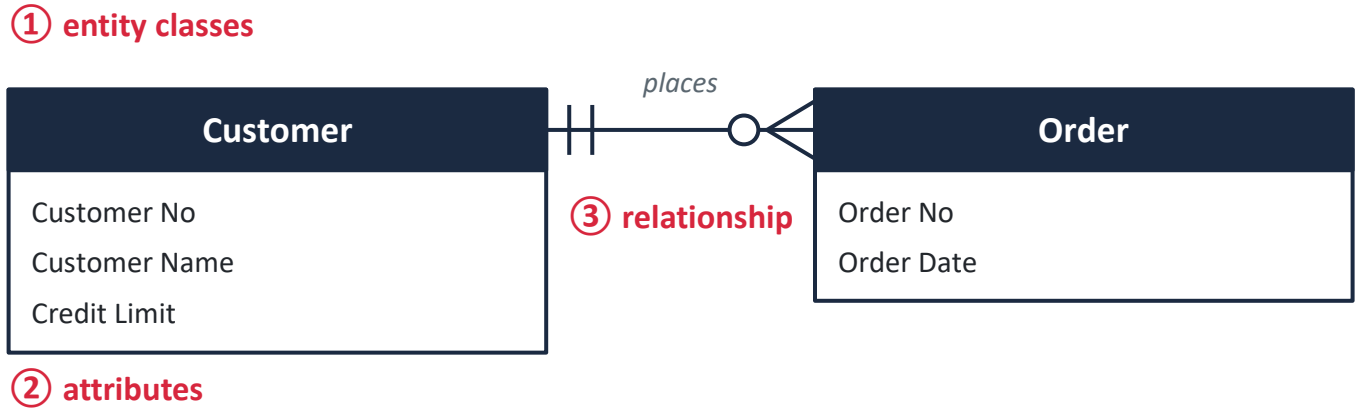


A data model is a design: the house-plan analogy

Building a house	Building a database
Architect	Data modeler
House plan	Data model
Client's needs and budget	Business requirements
Building codes	Normalization and design rules
Builder	Database administrator / developers
The house	The database

- **Nobody builds without a plan:** if no one designs the data, the structure still exists, just by accident.
- **Plans are cheaper to change** than buildings: fix structure on paper, not in production.
- **Plans communicate:** owners, builders and inspectors all read the same drawing.
- **Plans involve choice:** two architects give two good but different designs.

What does a data model contain?



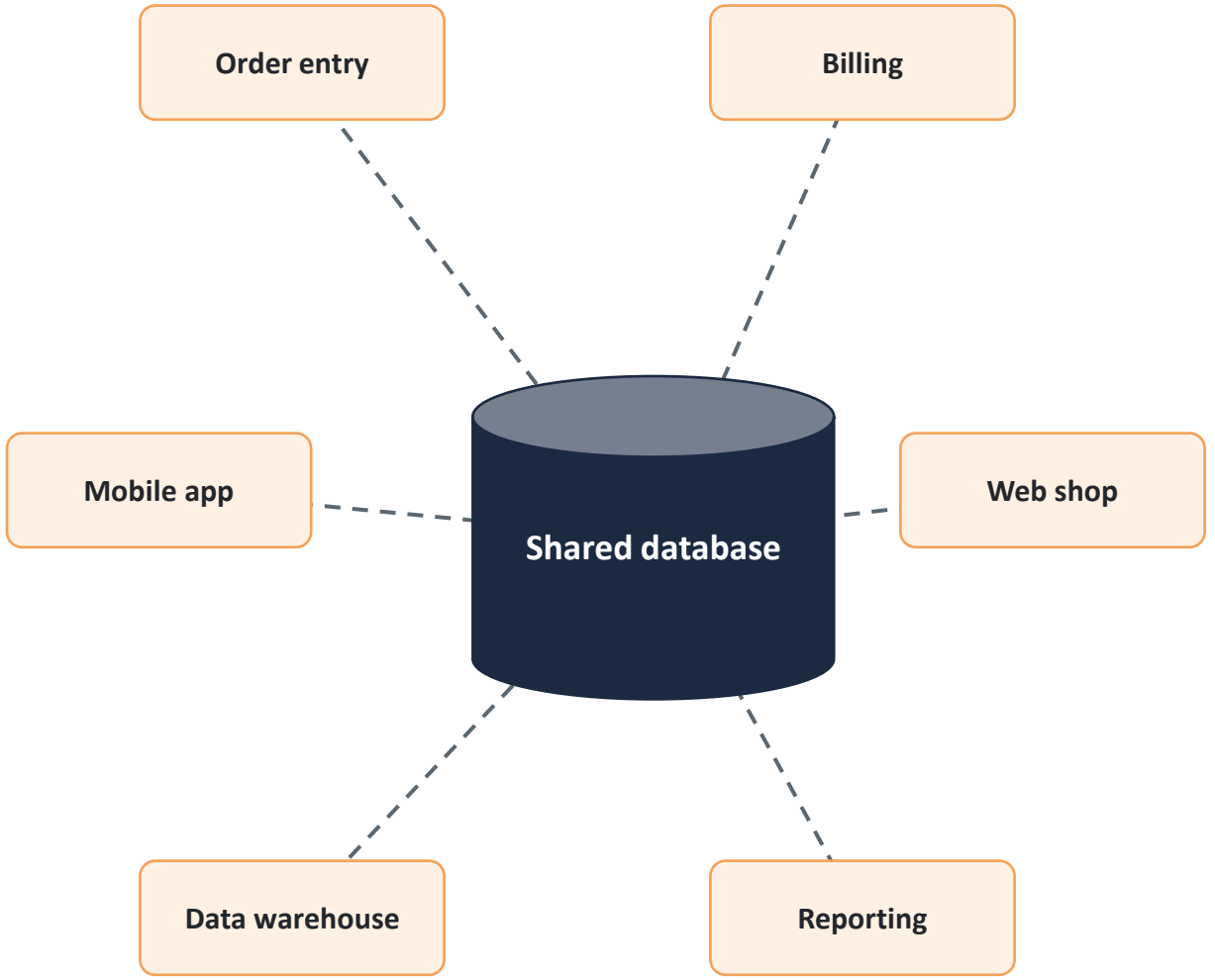
SUPPORTING DOCUMENTATION

④ **Definition:** *"Customer: a person or organization that has bought from us or holds an open quote."*
⑤ **Rule:** *"Every order must be placed by exactly one customer."*

1. **Entity classes**
   things the business keeps data about
2. **Attributes**
   facts about each entity class
3. **Relationships**
   how entity classes are associated
4. **Definitions**
   precise meaning of every item
5. **Business rules**
   constraints the data must obey

A data-centered perspective

- Programs (processes) come and go; the data they share usually lasts for many years.
- Many programs read and update the same data, so its structure affects all of them.
- Changing a table late means changing every dependent program, report and interface.
- **Conclusion:** design the data carefully and early; it is the foundation the rest of the system stands on.



Data versus processes

Characteristic	Processes (programs)	Data (database)
Typical lifespan	Rewritten or replaced every few years	Often outlives several generations of programs
Sharing	Usually serves one function	Shared by many programs and users
Cost of changing structure	Local to that program	Ripples to every program that uses it
What users see	Screens and reports	The facts behind every screen and report
Design implication	Can evolve incrementally	Worth getting right early

Business processes change often; the things a business keeps data about (customers, products, orders) change far less.

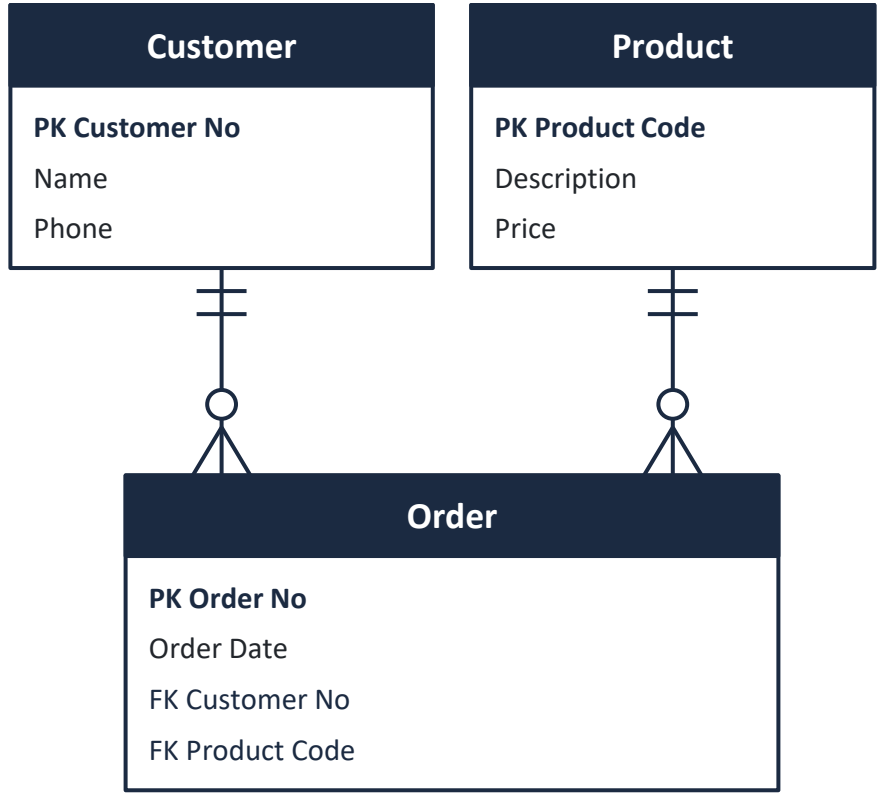
A simple example: one sheet vs. a structured model

Order No	Date	Customer	Cust Phone	Product	Price
1001	3 Mar	**Acme Ltd**	**555-0101**	**Widget**	**4.50**
1002	4 Mar	**Acme Ltd**	**555-0101**	Gadget	9.00
1003	4 Mar	Bolt Co	555-0199	**Widget**	**4.50**

PROBLEMS

What goes wrong with the single sheet

- Acme's phone number is stored twice, so a change must be made in several rows.
- The Widget price is repeated; two rows could disagree.
- Delete order 1003 and we lose the only record that Bolt Co exists.
- A new product cannot be recorded until someone orders it.



Each fact stored once

The same data, structured: three tables

Customer No	Name	Phone
C1	Acme Ltd	**555-0101**
C2	Bolt Co	555-0199

Product Code	Description	Price
P1	Widget	**4.50**
P2	Gadget	9.00
P3	Sprocket	2.25

Order No	Date	Customer No	Product Code
1001	3 Mar	C1	P1
1002	4 Mar	C1	P2
1003	4 Mar	C2	P1

✓ **Phone changes:** Update one row in Customer.

✓ **Price consistency:** The Widget price is stored once.

✓ **Delete order 1003:** Bolt Co remains in Customer.

✓ **New product:** Sprocket (P3) exists with no orders.

Still simplified: each order holds only one product. Chapter 3 shows how an Order Line resolves that.

CHAPTER 1 · PART B

B

Design, Choice and Creativity

There is usually more than one good model

IN THIS PART

- Modeling as design
- Two valid models
- A third option
- How to choose

Data modeling is a design activity

1 Choice and creativity

Data modeling offers opportunities for choice and creativity.

2 Many possible models

A problem usually has many models that meet the business requirements and follow the rules of sound design.

3 Criteria decide

Choosing the best model means weighing several criteria, whose importance varies from case to case.

4 No single answer

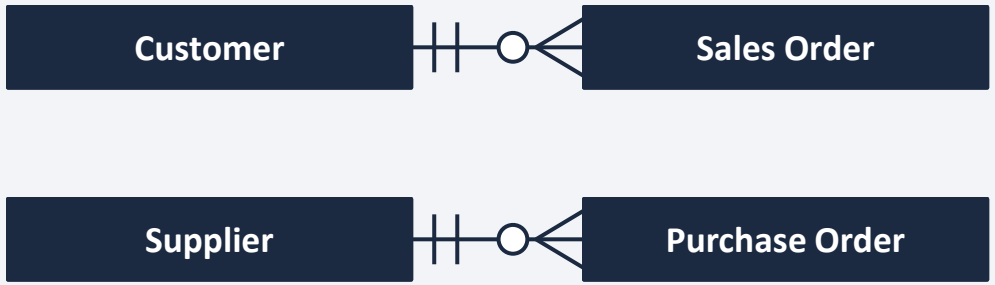
The emphasis is on understanding the merits of different solutions, not prescribing one "correct" answer.

Source: Preface, p. xxiii

Design, choice and creativity

Requirement: *"We buy from suppliers and sell to customers. Some organizations are both."*

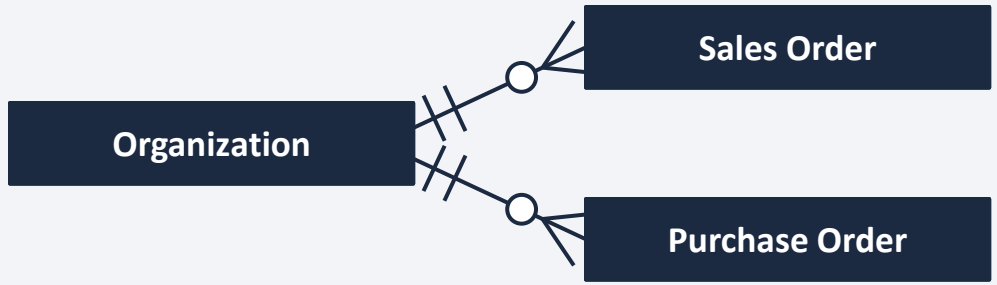
Model A: separate classes



+ Matches how the business talks; rules specific to customers or suppliers are easy to enforce.

– An organization that is both is stored twice, and its address must be updated twice.

Model B: one generalized class

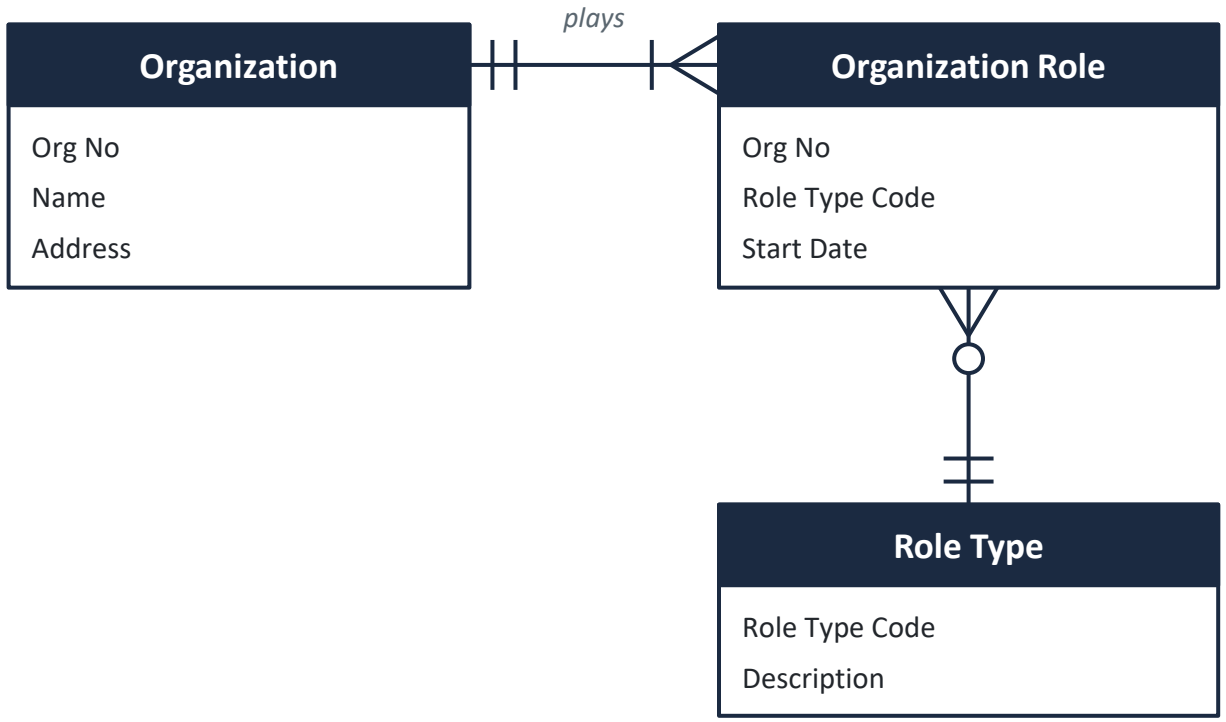


+ Each organization stored once; new roles (e.g., agent) are easy to add.

– Rules that apply only to customers or only to suppliers are less visible in the structure.

Both models are valid. The modeler chooses, using explicit quality criteria.

Model C: organization roles



Role Type values: Customer, Supplier, Agent, ...

Assessment

+ Organization stored once; a new role (e.g., Agent) is just a new row in Role Type, with no change to the structure.

– Rules such as "only customers have a credit limit" cannot be enforced by the structure; the model is more abstract for users.

Compared with Models A and B: most flexible, least rule-enforcing, hardest to explain.

How do we choose between valid models?

1

Understand requirements

What must the system record, and which rules matter?

2

Generate options

Propose more than one model; use patterns and experience.

3

Evaluate against criteria

Completeness, rules, flexibility, communication...

4

Decide and document

Record the choice and the reasons for it.

APPLYING IT

Model A enforces customer-only rules; Model C adapts to new roles without change. If the business adds new kinds of partner every year, flexibility may win. If credit rules are critical, Model A may win. The criteria decide.

CHAPTER 1 · PART C

C

Why Is the Data Model Important?

Three reasons: leverage, conciseness, data quality

IN THIS PART

- Leverage
- Conciseness
- Data quality

Why is the data model important?

1 Leverage

A small change to the model can have a large effect on the whole system: program logic, screens and reports all follow from the data structure.

Example Allowing "many addresses per customer" late in a project can touch dozens of programs.

2 Conciseness

A data model diagram summarizes on a page or two what would take many pages of written requirements, so it is an efficient way to review scope.

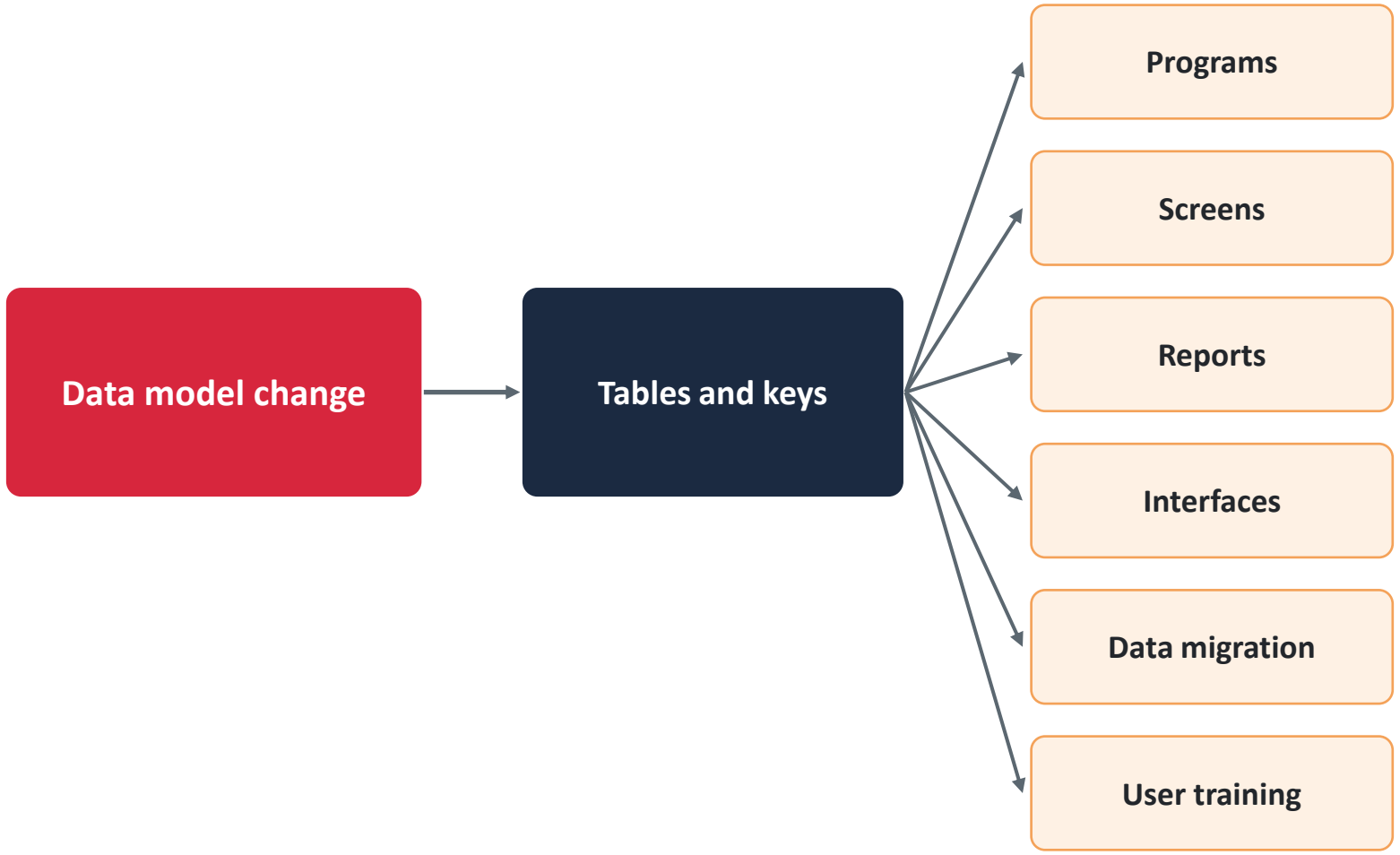
Example One diagram shows reviewers every kind of thing the system will remember.

3 Data quality

A good model prevents redundant, inconsistent data and defines exactly what each item means, which underpins data quality.

Example If "Customer" is defined precisely, two systems cannot count customers differently.

Leverage: small model changes, big system effects

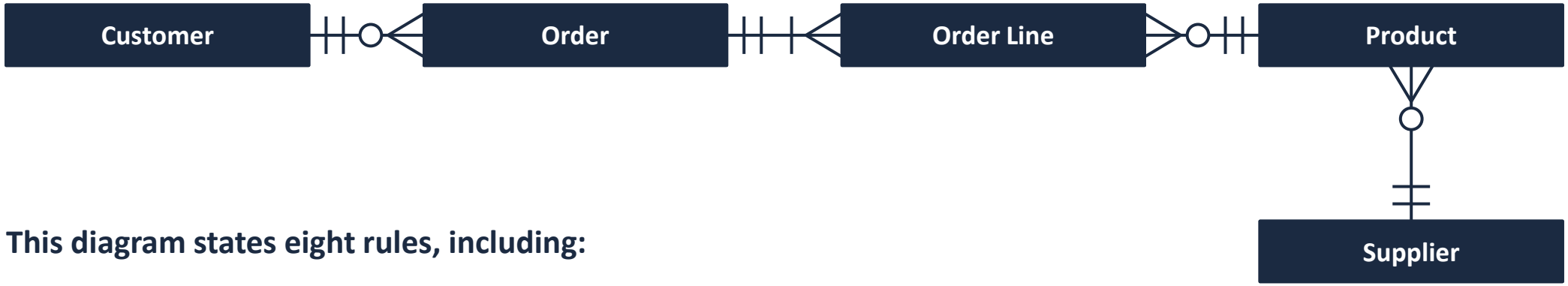


EXAMPLE

Requirement discovered late: *"A customer can have several delivery addresses."*
Address moves from Customer into a new table. Every program, screen and report that uses the address must change, and existing data must be migrated.

On paper, during modeling, the same change takes minutes.

Conciseness: a lot of meaning in one diagram



This diagram states eight rules, including:

- Each order must be placed by exactly one customer; a customer may place many orders.
- Each order must have at least one order line.
- Each order line is for exactly one product.
- Each product is supplied by exactly one supplier (a rule the business may want to challenge!).

Reviewers can check scope and rules quickly, and spot doubtful rules like the supplier one.

Data quality starts with the model

Design problem	What happens	Modeling remedy
Unclear definitions	Sales counts a "customer" as anyone with an account; Finance counts only those who paid this year. Reports disagree.	Define every entity class and attribute precisely.
Overloaded fields	A "Notes" field holds delivery instructions, complaints and phone numbers.	One fact per attribute; separate attributes for separate facts.
Redundant storage	A customer's address in three tables; only one gets updated.	Store each fact once (normalization, Chapter 2).
Missing rules	Orders exist with no customer because nothing prevents it.	Model the rule; enforce it with keys and constraints.

Poor structure produces poor data, however carefully people enter it.

CHAPTER 1 · PART D

D

What Makes a Good Data Model?

Eight criteria, their trade-offs, and performance

IN THIS PART

- Overview
- Criteria in detail
- Conflicting objectives
- Checklist
- Performance

What makes a good data model?

1

Completeness

Supports all the data the business requirements call for.

2

Nonredundancy

Each fact is recorded in only one place.

3

Enforces business rules

The structure reflects the rules, e.g., every order must have a customer.

4

Data reusability

Data can serve applications beyond the one it was built for.

5

Stability & flexibility

Copes with business change with little or no restructuring.

6

Elegance

Classifies data neatly and simply; concise and coherent.

7

Communication

Understandable to the business people who must verify it.

8

Integration

Fits with the organization's other databases rather than forming a silo.

Completeness and nonredundancy

1. Completeness

The model supports all the data required by the business processes and reports in scope. Test by walking through each requirement and asking "where is that stored?"

2. Nonredundancy

Each fact is represented only once. Redundancy wastes space, but the real danger is inconsistency when copies disagree.

INCOMPLETE

Requirement: "Report sales by region." The model has Customer and Order but nothing that records a region. The model is incomplete; add Region (or a region attribute) before building.

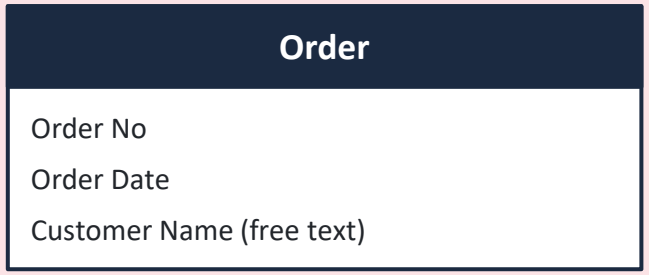
REDUNDANT

Order stores Customer Name as well as Customer No. When the customer changes name, old orders show the old name, new orders the new one. Which is right? Store the name once, in Customer.

Enforcement of business rules

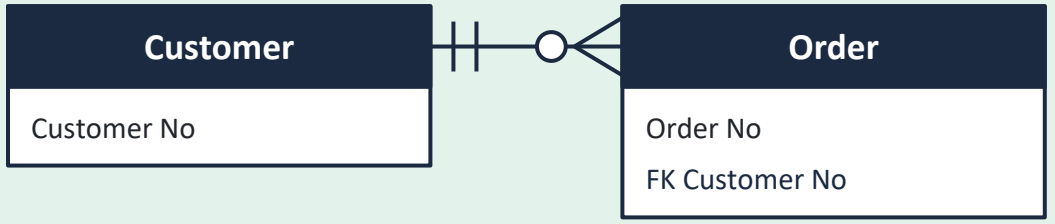
Rule: "Every order is placed by one registered customer."

Weak: rule not enforced



Anything can be typed: misspelled names, customers who do not exist, or nothing at all.

Strong: rule built into the structure



The mandatory relationship (implemented as a required foreign key) means the database rejects an order without a valid customer.

Data reusability

Shaped for one report

```
MONTHLY_SALES (Region, Jan Total, Feb Total,
..., Dec Total)
```

Useful for exactly one report. Weekly figures, sales by product, or next year's data need new structures.

Shaped around the business

```
SALE (Sale No, Sale Date, Product Code,
Customer No, Region Code, Amount)
```

The monthly report, a weekly dashboard and a product analysis can all be derived from the same data.

GUIDELINE

Record facts at the level they occur (each sale), and derive summaries from them. Summaries can always be recomputed; detail thrown away cannot be recovered.

Stability and flexibility

Stability

Handles changes in requirements without any change to the structure.

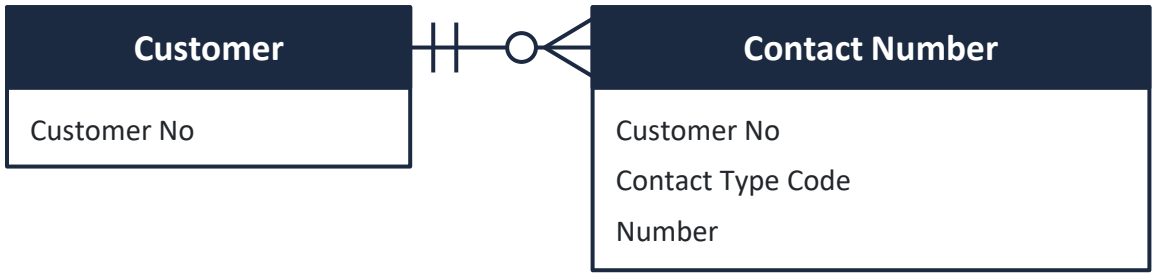
Flexibility

When a change is needed, it can be made by adding, not by restructuring.

Example: the business adds a new contact channel (e.g., a messaging app).

Customer
Customer No
Home Phone
Work Phone
Mobile Phone

Needs a new column, and every program that lists contact numbers must change: not stable.



A new Contact Type value; no structural change: stable.

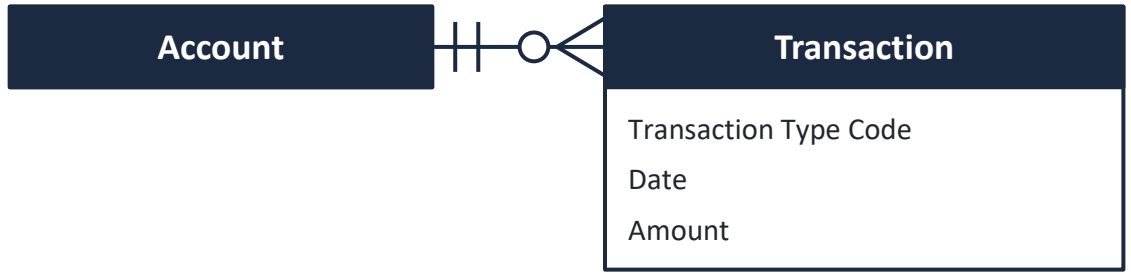
Elegance

Less elegant: one entity class per kind of event



Six near-identical structures, each with account, date and amount; six sets of programs; a new transaction type means a new table.

More elegant: one generalized class



One structure; types are data. Simple, compact, easy to extend.

CAUTION

Elegance is not the same as maximum generalization. If each transaction type had very different data and rules, separate classes (or subtypes, Chapter 4) might be the better design.

Communication and integration

7 Communication

- Use business names: "Policy Holder", not "PH_REC".
- Keep diagrams readable: sensible layout, limited size per page.
- Write definitions users can check.
- Read relationships as sentences with users (Chapter 3).

8 Integration

- Reuse existing definitions of shared concepts (Customer, Product).
- Align keys and codes with other systems where possible.
- Avoid creating another inconsistent copy of shared data.

SILO PROBLEM

Sales and Support each keep their own customer table with different numbers. Nobody can list all support calls from the top ten sales customers.

Conflicting objectives

Flexibility	vs	**Rule enforcement**	Generic structures absorb change but enforce fewer rules.
Elegance	vs	**Communication**	Abstract models are neat but harder for users to recognize.
Completeness	vs	**Time and cost**	Modeling every detail delays delivery; scope must be agreed.
Reusability	vs	**Simplicity for one application**	General structures need more work than one report-shaped table.

There is no model that maximizes every criterion. Decide what matters most for this system, and record why.

Quality review checklist

Criterion	Question to ask	Warning sign
Completeness	Can every requirement be met from this data?	A report needs data nobody stores
Nonredundancy	Is each fact stored exactly once?	The same value in several tables
Business rules	Does the structure enforce the key rules?	Important rules exist only in documents
Reusability	Could other applications use this data?	Tables shaped like one report
Stability & flexibility	What happens when the business changes?	New types need new columns or tables
Elegance	Is it simple and consistent?	Many near-identical entity classes
Communication	Can users read and verify it?	Technical names; no definitions
Integration	Does it fit with existing data?	A new copy of shared data

What about performance?

Conceptual

Ignore performance; model the business correctly.

Logical

Keep the normalized, correct structure.

Physical

Tune: indexes, storage, partitioning and, if justified, controlled denormalization.

- **Why wait?** Performance needs depend on the DBMS, volumes and usage, which are only known later.
- **Modern DBMSs** offer many tuning options that do not change the logical structure.
- **Denormalize with care:** measure first, document the redundancy, and control how copies are kept consistent.

EXAMPLE

A report totals order lines slowly. Before storing an "Order Total" (redundant data), try an index on Order Line (Order No). If a stored total is still needed, keep it consistent with a trigger or batch process, and document it.

CHAPTER 1 · PART E

E

Stages and Schemas

Conceptual, logical and physical models

IN THIS PART

- Three stages
- One example, three levels
- Deliverables
- Three-schema architecture
- Terminology

Conceptual, logical and physical models

Conceptual

What does the business need?

Contains: Entity classes, relationships, attributes, business rules

Scope: Technology independent

Audience: Business stakeholders & modelers

Logical

How does it map to a relational DBMS?

Contains: Tables, columns, primary and foreign keys, normalized structure

Scope: DBMS-type specific

Audience: Designers & developers

Physical

How will it run efficiently?

Contains: Indexes, storage, partitioning, controlled denormalization, views

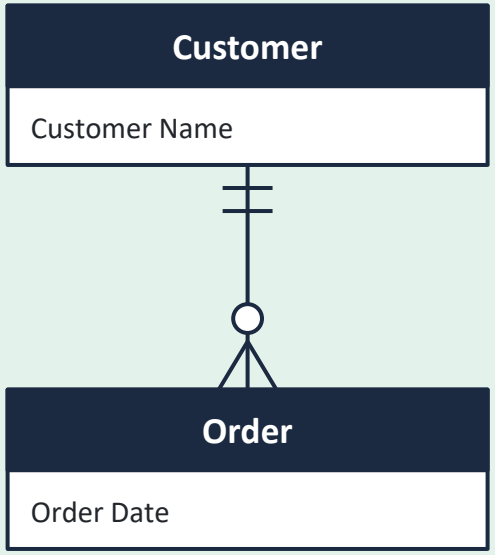
Scope: Product specific (e.g., Oracle, SQL Server)

Audience: Database administrators

Decisions flow left to right; changes made later should be reflected back in the earlier models.

One example at three levels

Conceptual



Business terms only; no keys or data types.

Logical (relational)

CUSTOMER (**Customer No**, Customer Name)

ORDER (**Order No**, Order Date, Customer No*)

Tables, columns, primary keys (underlined) and foreign keys (*).

Physical (SQL)

```
CREATE TABLE customer (
  customer_no INTEGER PRIMARY KEY,
  customer_name VARCHAR(100) NOT NULL);

CREATE TABLE sales_order (
  order_no INTEGER PRIMARY KEY,
  order_date DATE NOT NULL,
  customer_no INTEGER NOT NULL
    REFERENCES customer);

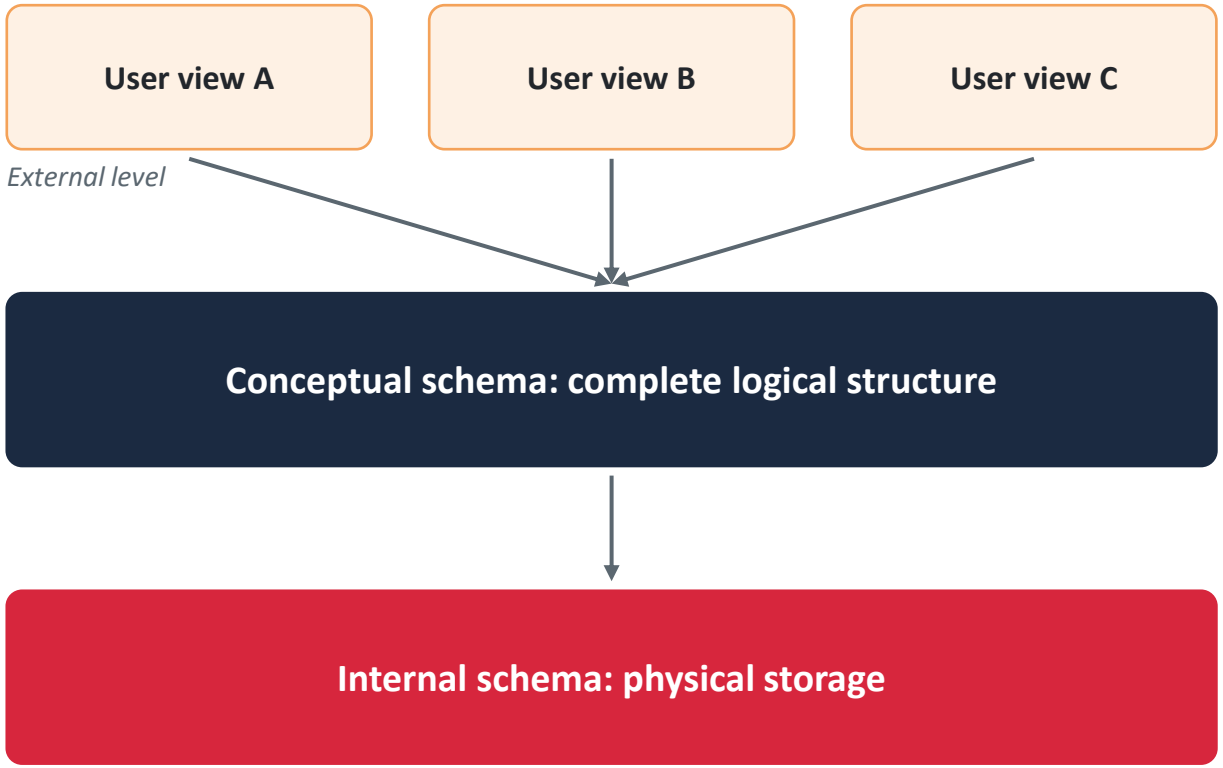
CREATE INDEX ix_order_cust
  ON sales_order (customer_no);
```

Product-specific: data types, index, renamed table (ORDER is an SQL reserved word).

Deliverables and decisions at each stage

	Conceptual model	Logical model	Physical model
Main question	What data does the business need?	How is it organized for this kind of DBMS?	How is it stored and tuned on this product?
Typical content	Entity classes, relationships, attributes, definitions, rules	Tables, columns, primary and foreign keys, nullability	Indexes, storage, partitions, views, denormalization
Key decisions	Level of generalization; scope	Keys; implementation of subtypes and relationships	Performance trade-offs
Reviewed by	Business stakeholders	Designers, developers	DBAs
Depends on technology?	No	DBMS type (e.g., relational)	Specific product and version

The three-schema architecture



ANSI/SPARC architecture (mid-1970s)

- **External schemas:** the views that particular users or programs see.
- **Conceptual schema:** the single, complete logical description of the data.
- **Internal schema:** how the data is physically stored and accessed.
- **Benefit:** storage can change without affecting views, and vice versa.

TERMINOLOGY TRAP

ANSI's "conceptual schema" corresponds to what we call the logical data model. Our "conceptual model" is a business-level model that sits above it.

Terminology: matching the words to the levels

Conceptual (E-R)	Logical (relational)	Relational theory	Physical / SQL
Entity class	Table	Relation	Table
Entity (instance)	Row	Tuple	Row / record
Attribute	Column	Attribute	Column / field
Relationship	Foreign key	Foreign key	Foreign key constraint
Identifier	Primary key	Primary key	Primary key constraint + index

NOTE

The book uses "entity class" for the category (Customer) and "entity" or "instance" for one member (Acme Ltd). Other texts say "entity type" and "entity".

CHAPTER 1 · PART F

F

Data Modeling in Practice

Methods, people and relevance today

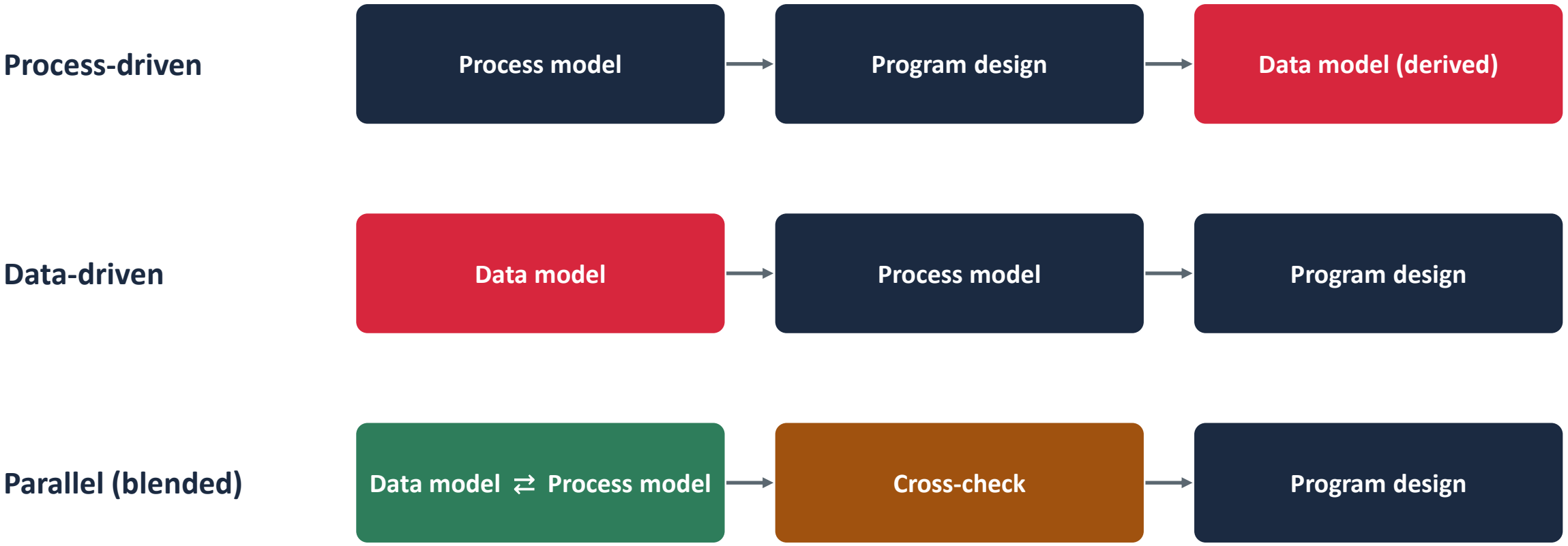
IN THIS PART

- Development approaches
- Who is involved
- Costs & benefits
- Still relevant?
- Alternative approaches
- Summary

Where do data models fit in development?

Approach	Role of the data model	Watch out for
Process-driven	Derived from process requirements, often late	Data shaped around today's processes; poor reuse
Data-driven	Developed first; processes designed around it	Delays if the model is over-engineered
Parallel (blended)	Developed alongside the process model and cross-checked	Needs close coordination between modelers
Object-oriented	Class model combines data and behavior	Class diagrams may not map cleanly to tables
Prototyping	Model evolves with each prototype	Structure can grow by accident; review it deliberately
Agile	Built incrementally, "just enough" each iteration	Database refactoring is costly; get the core right early

Process-driven vs. data-driven: sequence



Whatever the approach, a data model is produced: the question is whether it is designed deliberately or emerges by accident.

Object-oriented, prototyping and agile approaches

Object-oriented

Class models combine data and behavior. The class diagram often doubles as the data model.

Watch out: Classes designed for program structure may not suit a shared database; a separate data view is still needed.

Prototyping

The model evolves as working prototypes are shown to users.

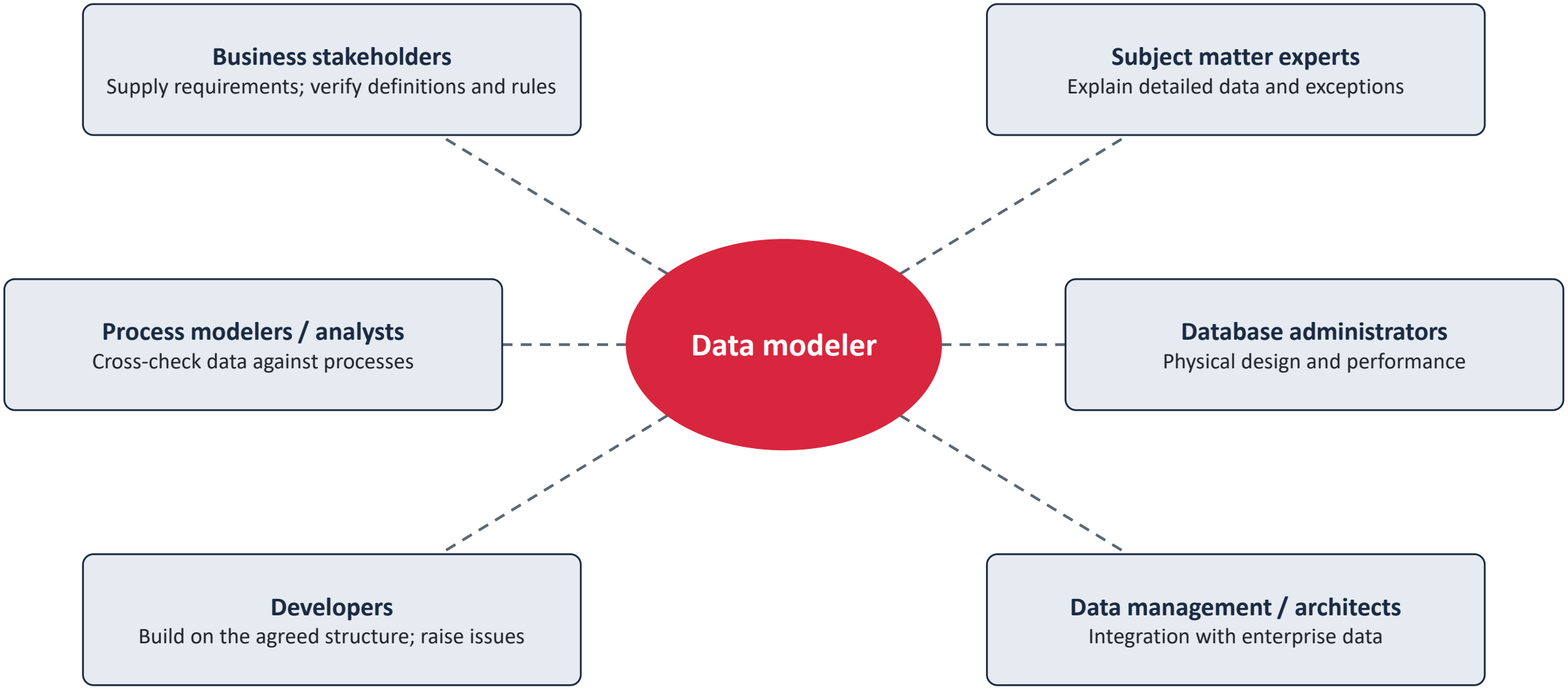
Watch out: Quick fixes accumulate; review the structure deliberately before the prototype becomes the product.

Agile

Requirements and design are developed incrementally in short iterations.

Watch out: Code is easy to refactor; a database full of data is not. Model the core entity classes early.

Who should be involved in data modeling?



Costs and benefits of data modeling

Costs

- Time of skilled modelers and business experts.
- Workshops, reviews and documentation.
- Tools and training.
- Perceived delay before "real" development starts.

Benefits

- Structural errors found on paper, not in production.
- Less rework in programs, screens and reports.
- Better data quality and consistent meaning.
- Easier integration, reporting and future change.

Every database has a model. Not modeling does not avoid the cost; it just postpones it to when change is most expensive.

Is data modeling still relevant?

Packaged software

A model of your own requirements lets you check whether a package fits, and plan customization and interfaces.

Data integration

Connecting systems requires understanding and mapping each one's data.

Data warehouses

Data from many sources must be brought into one designed structure (Chapter 16).

Personal computing

Spreadsheets and desktop databases built by users suffer from the same structural problems.

XML and messages

Data exchanged between systems still needs a sound, agreed structure.

Summary

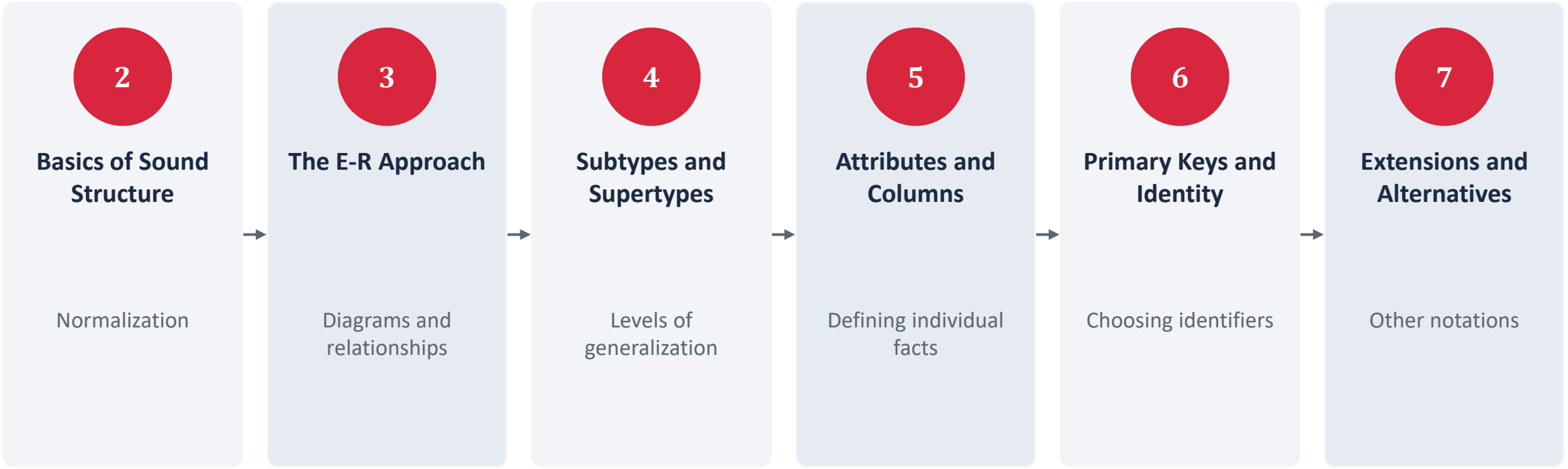
Wherever data is stored or exchanged, someone is making modeling decisions.

Alternative approaches to data modeling

Approach	Main idea	Where you meet it
E-R with crow's foot (this course)	Boxes for entity classes, lines for relationships	Most data modeling tools; industry practice
Chen E-R	Rectangles, diamonds for relationships, ovals for attributes	Academic textbooks
UML class diagrams	Classes with attributes and multiplicities	Object-oriented software projects
Object Role Modeling	Facts expressed as roles objects play in sentences	Specialist tools; conceptual analysis

The concepts (entity classes, attributes, relationships, rules) transfer between notations; learn them once, then any notation.

Where to from here? Part I of the book



Part II then follows a real project from planning to physical design; Part III covers advanced topics.

Chapter 1 summary

1 A data model is a design

Every database has one; the choice is whether to design it deliberately.

2 Data comes first

Data is shared and long-lived, so its structure affects everything.

3 Three reasons it matters

Leverage, conciseness and data quality.

4 Eight quality criteria

They often conflict; make trade-offs explicit.

5 Three stages

Conceptual → logical → physical; performance belongs in physical.

6 Always relevant

In any method, and for packages, integration, warehouses and XML.

Check your understanding

1. Why does the book call data modeling a design activity?
2. Give an example of "leverage": a small model change with a large system effect.
3. Which two quality criteria conflict in a generic "Organization Role" model?
4. At which stage should performance be addressed, and why?
5. What does ANSI's "conceptual schema" correspond to in our terminology?
6. Name two situations where modeling matters even if software is bought.

Answers

1. Several valid models can satisfy the same requirements; the modeler chooses among them using criteria.

2. Allowing several addresses per customer late in development changes programs, screens, reports and data.

3. Flexibility (new roles need no change) vs. enforcement of business rules (customer-only rules are lost).

4. Physical design: it depends on the DBMS, volumes and usage, and should not distort the business model.

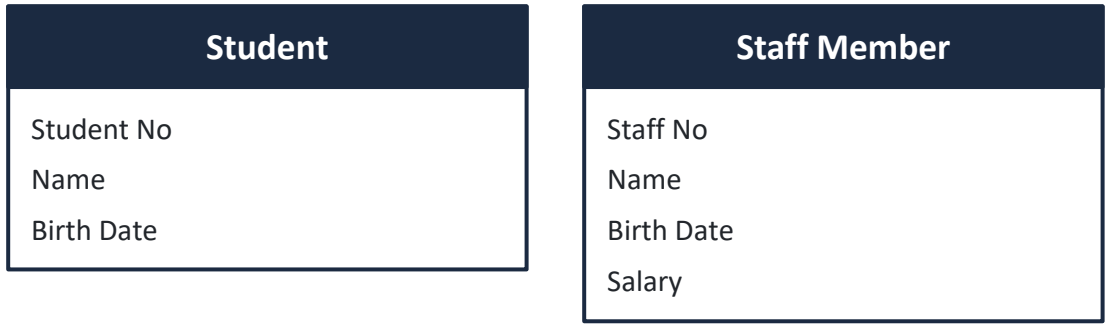
5. The logical data model; our "conceptual model" is a higher, business-level view.

6. Evaluating package fit, data integration, data warehouses, interfaces (e.g., XML messages).

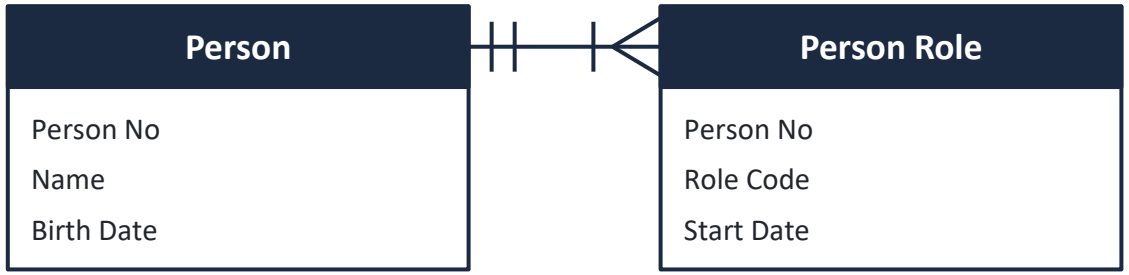
Exercise: evaluate two models

Scenario: A university records staff and students. Some students also work as tutors. The university may later add visiting researchers and alumni.

Model 1



Model 2



Tasks: (1) Rate each model on the eight criteria. (2) Which handles the student-tutor best? (3) Which is easier to explain to the registrar? (4) Which would you recommend, and why?

Exercise: discussion points

Criterion	Model 1 (separate classes)	Model 2 (person + roles)
Nonredundancy	✗ A student-tutor is stored twice (name, birth date)	✓ Each person stored once
Stability & flexibility	✗ Researchers and alumni need new entity classes	✓ New roles are new code values
Business rules	✓ Salary only for staff, enforced by structure	✗ Role-specific rules need extra constraints
Communication	✓ Matches how the registrar talks	? More abstract; needs explanation
Elegance	? Simple but duplicated attributes	✓ Compact and consistent

CONCLUSION

There is no single right answer. Given expected new roles, many would choose Model 2; subtypes (Chapter 4) can combine the strengths of both.

END OF CHAPTER 1

Questions for discussion

1. Think of a system you use. What would happen if its data structure had to change tomorrow?

2. Which of the eight quality criteria matters most for a hospital records system? For a marketing campaign tool?

3. Is a spreadsheet you keep for personal use "a database with a data model"? Why?

Next: Chapter 2, Basics of Sound Structure (normalization).